IE 643: Deep Learning - Theory and Practice **July-Dec 2026**

Project Report: Probing Video Diffusion U-Nets for Content vs. Motion

Team Name: Claudn't *Team Members: [roll numbers]*

Abstract

A text-to-video diffusion U-Net alternates spatial layers, which see one frame, with temporal layers, which see all frames. We asked where inside such a model the content of a clip (what is depicted) and its motion (how it moves) are actually decided, and whether that answer is good enough to edit one without the other. We used AnimateDiff (Stable Diffusion 1.5 with a motion adapter) and generated 360 clips in 750 controlled pairs in which a single prompt word changes and the initial noise is fixed. From these pairs we built selectivity maps over 16 layers and all 25 denoising steps, and tested three hypotheses whose test statistics were fixed before the data existed. Motion selectivity is weak almost everywhere; it peaks in the temporal module of the first decoder block (`up_blocks.0`, 8×8 latent) in 1000 of 1000 bootstrap resamples, and only during the first 2–3 of 25 steps (changepoint $\tau = 2$, 95% CI [1,3]). We then turned the static/dynamic split used for the maps into an intervention, a *component patch* that transplants only the frame-varying part $\tilde{x}$ or only the frame mean $\bar{x}$ of an activation between two generations. On 12 held-out test pairs the selected component patches beat a prompt-only edit on the median in both directions (median ΔD +0.041 and +0.068) and beat whole-tensor patching, but their 95% confidence intervals include zero, so our pre-registered success criterion is not met. Temporal-attention injection, the training-free baseline, is the only method whose interval excludes zero: +0.105 (CI [0.001, 0.204]) over the first third of the steps, and +0.120 (CI [0.020, 0.203]) over all steps. We also trained motion LoRAs placed by the map and in a deliberately wrong place. Across three seeds the map-guided LoRA is the only one with a positive median gain in every seed, though the gain is small (+0.009 to +0.015) and not significant. We also built a Gradio interface for live edits, uploaded videos and browsing every scored clip.

1 Introduction

Recent text-to-video diffusion models can easily turn a prompt into a short, realistic clip. Every such clip combines two things i.e. its *content* which denotes what is present in the frames, like a dog etc. , and its *motion* which denotes how the content present in the frames is moving. In principle the two are independent in the sense that the same motion could be performed by a different object. For example instead of a dog running along a road you could have a horse running along the road. This project asks that at whether a video diffusion model keeps them apart internally and if it does then in which layers and at which denoising steps does it decide content and and ata which steps does it decide the motion, and whether that knowledge is precise enough to change one while keeping the other.

Answering this question is important because people increasingly want to edit a generated clip rather than regenrating it because changing the prompt changes the subject, the background and motion all at once. Recent video diffusion models are built on image diffusion models by adding temporal layers to them, as AnimateDiff [10] does with Stable Diffusion [28]. Recent editing methods [34; 38; 24; 35] intervene on chosen layers or features to transfer or customise motion, but they chose those places by design or by ablation. The obvious reading by archtecture stating spatial and temporal layers is usually assumed rather than measured. A measured mapping of where the content lies and where the motion lies would allow targetted edits, changing only certain parts of the clip and retaining the other desired ones rather than entire regeneration of the clip. It would show us where to place the few trainable parameters of LoRA [17] when

teaching the model a new motion. A negative answer is also useful because if the split is not clean then the editing methods that assume this clean split are working for some other reasons that need further invetsigation.

To find such a map, we experiment with the model. The model is a U-net architecture that turns noise into video over a sequence of denoising steps. It was trained on just the input and the output and was never told as to which part of the model should carry the motion and which should carry the content. So any such seperation should be something that it learned on its own and has to be located emperically because it is spread across many high-dimensional layers and across many denoising steps so it cannot be read by weights alone nor by the architecture diagram. Architectural diagrams labelling layers as spatial and temporal tells us what each layer *can* see not what it actually ends up encoding. Finding the seperation would require us to actually study the trained network by changing the input in a controlled way and then observing how the activations are responding to those changes and then testing whether the places we found really control the desired properties or by intervening on them midst generation. We then use evalutations to judge whether the edits were successful or not.

The report is organised as follows. Section 2 lists how the work progressed. Section 3 surveys the work we built on. Section 4 describes the methods, split into work done before and after the prep-presentation review. Section 5 covers the data, Section 6 the experiments and their settings, and Section 7 the results. Section 8 gives the plan for novelty assessment and Section 9 concludes.

2 Project Workflow

We started with the question of finding out where the content lives and where the motion lives. The architecture suggests an easy answer that the appearance would be stored in the spatial layers and the motion would be stored in the temporal layers. But we wanted to test it out by measuring it. Our first experiments showed that the motion was harder to measure than we expected because pixels that move also change depending on the object performing the motion. We developed tools to look inside the model when it generated clips and used those measurements to gauge where is content stored and where is motion stored. And based on those observations we tried to edit the model mid generation to validate our hypthesised places of residence of content/motion. Some of our predictions were correct and some of them turned out to be false that told us what to try out next.

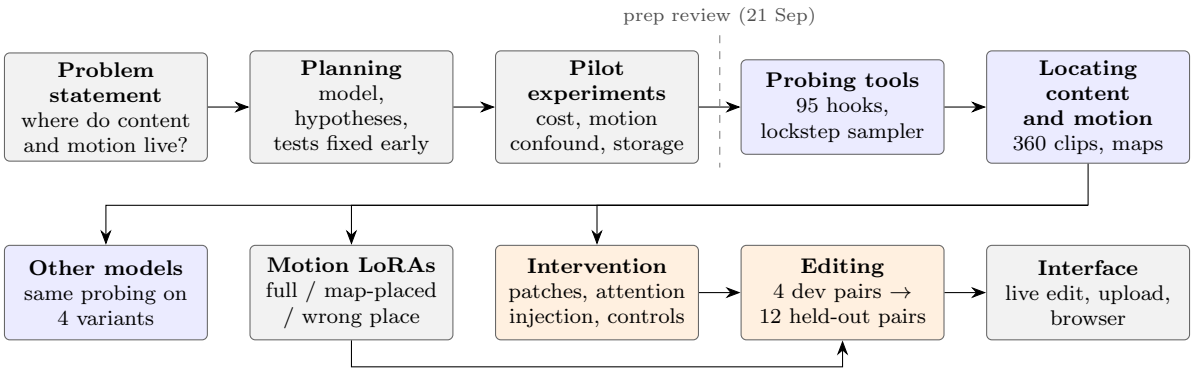


Figure 1: Order of the work. Everything left of the dashed line was done before the prep-presentation review. The maps from the probing decide where we intervene and where the map-placed LoRA sits, and the same probing is then repeated on four other model variants.

- **Problem framing:** Problem was to find out which layers store content and which of them store

motion of a video diffusion U-net model. Then use this information to edit a clip so that one of these is kept intact and the other is changed.

- **Planning:** We defined what content meant and what motion meant so we could measure it properly, chose AnimateDiff as our model over ModelScope and Stable Video Diffusion because it has explicit spatial and temporal split and takes text prompts as input and we fixed our prompt, the variables we would change and three hypothesis that we would test before seeing any data.
- **Experiments:**
  - Measured the cost of generating each clip
  - Found that the pixel space motion also depends on the object we are using to perform the motion which gave us the idea to measure motion on the residual of the frame mean for each frame
  - Found out that storing the full activations required way too much memory so we scored the pairs during generation itself.
- **Probing Tools:** Built a package that hooks into 95 modules of the U-net, wrote a sampler that generated groups of clips thata were bit identical to the diffusers pipeline and built a runner for the Kaggle T4s.
- **Locating content and motion:** Generated 360 clips that formed 750 pairs with the one word swaped in the prompt. Then selectivity mas were created for each layer and steps. We tested Our 3 hypothesis and trained the linear probes.
- **Intervention Framework:** Built patches that transplant only the static or only the frame varying part of the activation from one generation to another. Also built a temporal attention injection baseline.
- **Editing:** Chose one configuration per direction on 4 pairs of development and scored it on 12 pairs. Added attention level edits and prompt switching.
- **Trained Model:** Trained motion LoRAs on where the map points and also where it does not. After finding scene cuts in training clips we retrained.
- **Check findings on other models:** We repeated the probing on 4 other model variants to see if the map is specific to the diffusion model we used or does it apply in general.
- **Interface:** A live app where we can edit an uploaded video and view the maps and cross-attention

3 Literature Survey

Diffusion models [14] generate data by learning to undo a gradual noising process. A clean sample z_0 is corrupted over T steps, and the noisy sample at step t has the closed form

$$z_t = \sqrt{\bar{\alpha}_t}\, z_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I), \tag{1}$$

where $\bar{\alpha}_t$ falls from 1 to nearly 0, so that z_T is pure noise. A network ϵ_θ is trained to predict the noise that was added,

$$\mathcal{L} = \mathbb{E}_{z_0,\epsilon,t} \left\| \epsilon - \epsilon_\theta(z_t, t, c) \right\|_2^2, \tag{2}$$

and a new sample is generated by starting from z_T and removing the predicted noise one step at a time; DDIM [30] does this deterministically in a few tens of steps. Latent diffusion [28], the model behind Stable Diffusion, runs this process in the $4 \times 64 \times 64$ latent space of a frozen VAE instead of on pixels, and

conditions on a prompt c encoded by the CLIP text encoder [27] (Figure 2). Classifier-free guidance [15] strengthens the prompt at sampling time by extrapolating away from the unconditional prediction with a guidance scale w,

$$\hat{\epsilon} = \epsilon_\theta(z_t, t, \emptyset) + w\,\big[\epsilon_\theta(z_t, t, c) - \epsilon_\theta(z_t, t, \emptyset)\big]. \tag{3}$$

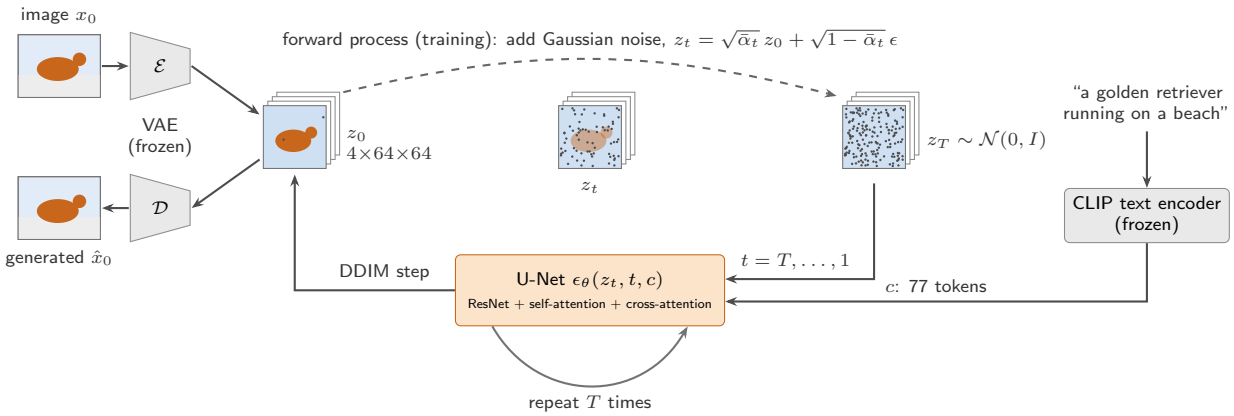


Figure 2: Latent diffusion [28], the base of AnimateDiff. Training adds noise to the VAE latent z_0 (Eq. (1)) and trains the U-Net to predict it (Eq. (2)). Sampling starts from noise z_T and applies the U-Net, conditioned on the CLIP text embedding c, for T DDIM steps before the VAE decodes the result.

The denoiser ϵ_θ is a U-Net [29]: an encoder that lowers the resolution of the latent from 64×64 to 8×8, a bottleneck, and a decoder that raises it back, with skip connections between blocks of the same resolution (Figure **??**). Each block has ResNet layers followed by transformer layers built on attention,

$$\mathrm{Attn}(Q, K, V) = \mathrm{softmax}\big(QK^\top/\sqrt{d}\big)\,V, \tag{4}$$

with the queries Q computed from the image features. In self-attention the keys and values come from the same features, so every position in the image looks at every other; in cross-attention they come from the 77 text tokens, which is how the prompt enters the image.

A video adds a frame axis. Video Diffusion Models [16] extended the image U-Net by factorising attention into a spatial part, where the tokens are the positions inside one frame, and a temporal part, where the tokens are the frames at one position. Video LDM [5] inserted such temporal layers into a frozen Stable Diffusion and trained only the new layers. AnimateDiff [10] takes this furthest. Many fine-tuned copies of Stable Diffusion 1.5 exist, each drawing a particular style, and the authors train one motion module that can be attached to any of them without retraining. Training has three stages (Figure 3a): a domain adapter, a LoRA [17] on the image layers that absorbs the blur and watermarks of web video and is switched off at inference; the motion modules, the only weights trained on video; and an optional MotionLoRA that teaches a new camera motion from 20–50 reference videos. A motion module (Figure 3b) reshapes the activation $x \in \mathbb{R}^{B\times C\times F\times H\times W}$ so that the F frames at each position form a sequence, applies temporal self-attention (Eq. (4)) with a sinusoidal position encoding, and adds the result back to its input,

$$z = x + W_{out}\,\mathrm{TempAttn}(x), \qquad W_{out} = 0 \text{ at initialisation}, \tag{5}$$

so that training starts from the unchanged image model.

We chose AnimateDiff because its content and motion weights are physically separate and were trained at different times: if even this model mixes the two, models trained jointly will too. Two features of the design shape our method. Because the motion module adds its output to its input, the activation after

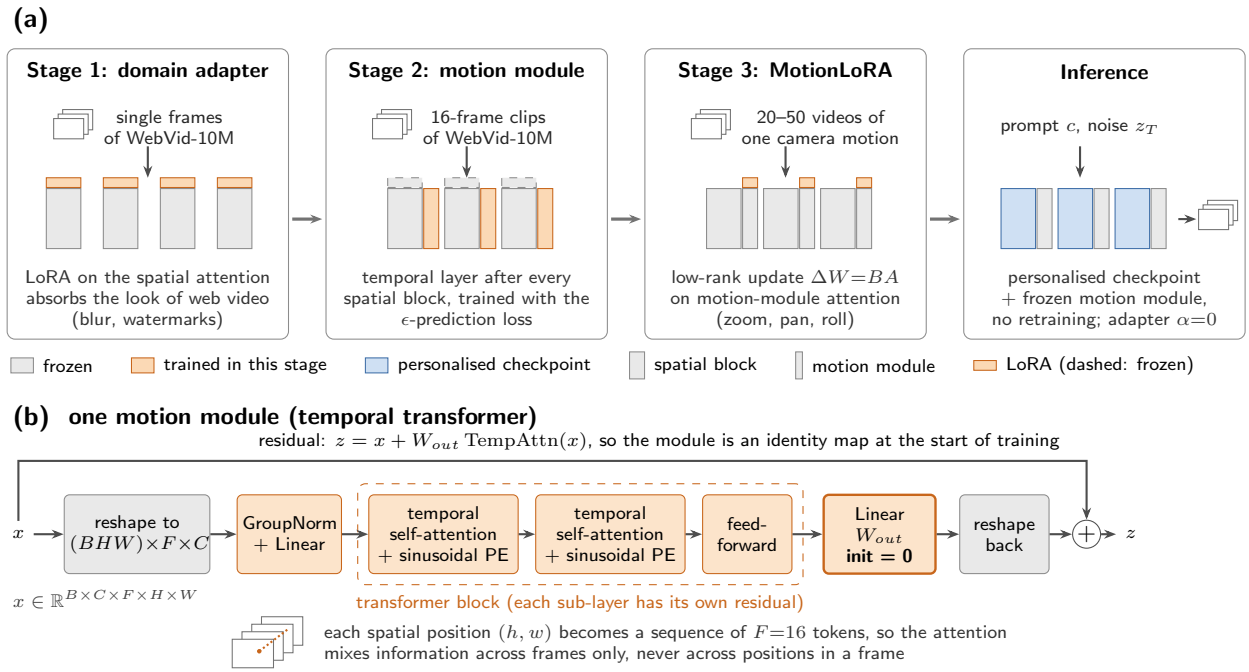


Figure 3: AnimateDiff [10]. (a) The three training stages and inference: a domain adapter (LoRA on the frozen image U-Net), the motion modules (the only weights trained on video), and MotionLoRA; at inference the motion module is attached to a personalised checkpoint. (b) One motion module: the activation is reshaped so that the F frames at each spatial position form a sequence, passed through temporal self-attention, and added back to its input through a zero-initialised output layer.

it holds content and motion together, so copying it from one generation into another copies both. And because the module already treats the frames as a sequence, it is natural to split an activation into a part that is the same in every frame and a part that changes from frame to frame. Knowing which layers are temporal, however, tells us only where motion could be computed, not what those layers hold; for that we needed a way to read motion out of the features, which MOFT gave us.

Xiao et al. [35] found that features of video diffusion U-net mix content and motion with most of the channels carrying content. They extract motion in two steps: first they subtract the mean over the frames $X - \frac{1}{F}\sum_{i=1}^{F} X_i$, which removes the part the remains constant over the frames, and then keep the top 4% channels chosen by PCA. The result is called MOFT and it encodes the direction of motion. They read it after the decoder block and use it to guide the generation of a new video without training by guiding the latents untill their MOFt matches a refrence. Our split is thier first step and our hypothesis comes from this paper. Our component patch copies the frame-varying part of the activation from one generation to another. MOFT shows where the motion can be read from the features but not whether the model actually uses it there. To test that we use activation patching.

To test whether the model uses a feature, one has to intervene on it, and the standard tool for this in interpretability is activation patching. Meng et al. [25] introduced it as causal tracing to find where GPT stores facts (Figure 4a). They run the model on a prompt such as "The Eiffel Tower is in", corrupt the subject's embeddings with Gaussian noise so that the answer is lost, and then restore one hidden state $h_i^{(\ell)}$ (token i, layer ℓ) to its value from the clean run. The indirect effect of that state is how much of the answer comes back,

$$\mathrm{IE}\big(h_i^{(\ell)}\big) = P_{*,\,h_i^{(\ell)}\leftarrow\mathrm{clean}}[o] - P_*[o], \tag{6}$$

where $P_*[o]$ is the probability of the correct answer o in the corrupted run. Basu et al. [3] applied the same procedure to Stable Diffusion, restoring one U-Net layer at every denoising step. Zhang and Nanda [37] showed that the conclusions depend on choices that are easy to make carelessly; in particular, Gaussian noise pushes the model off its input distribution, and they recommend replacing a key token with a related one instead. Heimersheim and Nanda [11] added that the direction matters: patching from the clean run into the corrupted one shows whether a state is *sufficient* to restore the behaviour, the reverse whether it is *necessary* to keep it, and the two can disagree when a network has redundant paths. We follow both. Our pairs differ in one word and share the same starting noise, so neither run is off its distribution; we copy activations at a chosen block over a window of denoising steps (Figure 4b) and patch in both directions. Patching a whole activation, however, moves everything a layer carries, while the editing literature intervenes more selectively, inside attention.

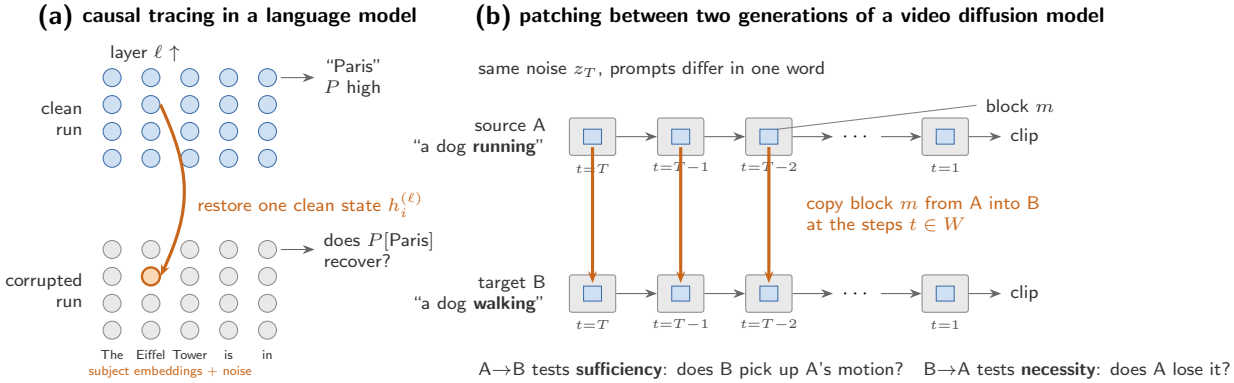


Figure 4: Activation patching. (a) Causal tracing [25]: a state from the clean run is restored in the corrupted run, and the recovered probability of the answer is its indirect effect (Eq. (6)). (b) The same idea for a video diffusion model: two generations from the same noise, whose prompts differ in one word, and an activation copied from one into the other at block m over a window W of denoising steps.

Image editing methods showed that the attention of Eq. (4) can be shared between two generations to keep one property while changing another. Prompt-to-Prompt [12] reuses the cross-attention maps of one generation in another, so that an edited prompt keeps the original layout. MasaCtrl [6] keeps appearance instead: after the first few steps, the target's spatial self-attention in the decoder queries the keys and values of the source,

$$\mathrm{Attn}\big(Q_{tgt},\, K_{src},\, V_{src}\big), \tag{7}$$

so the target takes the source's textures while its own queries keep its own layout. For video, MotionClone [24] found that the temporal attention maps of a reference video carry its motion, and guides the temporal attention of the generated video towards them with gradients, using a sparse mask that keeps only the strongest link between frames; Diffusion Motion Transfer [36] does the same with a loss on space-time features. UniEdit [2] combines both operations without any training: it edits motion by injecting temporal attention from a branch that generates the new motion, and keeps appearance by injecting spatial self-attention values from a branch that reconstructs the source video. Our attention-injection baseline is the simplest form of these ideas: in each temporal attention layer the target uses the source's attention probabilities with its own values,

$$\mathrm{softmax}\big(Q_{src}K_{src}^{\top}/\sqrt{d}\big)\, V_{tgt}, \tag{8}$$

replacing the probabilities directly instead of guiding towards them, since a backward pass through the U-Net at every step was beyond our compute budget. Our later attention-level edits reuse MasaCtrl's key–value injection for appearance and UniEdit's idea of taking appearance and motion from two different sources.

Two further works shaped specific choices. Our timestep hypothesis H2 follows the common observation that diffusion models settle layout and motion in the early denoising steps and appearance later, which Baherwani et al. [1] measured for video diffusion as two regimes with a fairly consistent boundary between them. Our trained model follows MotionDirector [38], which adapts a video model to a reference motion with a dual-path LoRA [17], one path on the spatial layers for appearance and one on the temporal layers for motion; we keep only the temporal path and ask whether its placement can come from our maps.

4 Proposed Approach

Our approach has three steps. First is that we generate pairs of clips from the same seed whose prompts differ only at one word and record how much does activations at each layer and step respond to this change. This gives us a map of where the content and motion are stored. Second we intervene on the model mid generation by copying activation or attentions from one generation to another at the places the map points to and see if we can see the change in the property that we intended to see in. Third we use the map for two edits that is transferring the motion or content between clips and deciding where to place small adapters that teach the model new motions. Apart from the LoRA and the probes, we have trained nothing and are working on pre trained work.

Our work is different from the existing work because:

- The existing editing methods like MotionClone and MOFt etc pick the layers to edit either by design or by ablation. We are first building a map to measure which layers contain what from controlled pairs. Then we test whether the place we motion is stored is actually the place where editing will work? In our results they are not the same place and that gap is important to understand.
- MOFT uses frame mean residual and PCA channels to guide the motion feature at a block. We apply the split to every coordinate and step (S_{dyn} vs S_{stat}) and then turn it into a component patch that transforms only one part.
- Causal Tracing (ROME) and its Stable Diffusion versions sstudy objects in a single frame whereas we use minimal pairs and patch in both directions to see if the place we found is actually neccessary and sufficient to control the property we are interested in.
- We place the LoRA by where the map tells to put it and compare it with an equal size LoRA placed where the map says motion is not present. This is different from MotionDirector which places the LoRA by design.

4.1 Network Architecture

AnimateDiff = frozen VAE + frozen CLIP text encoder + frozen SD1.5 U-Net + motion modules + DDIM sampler

- **VAE(frozen, about 84M)** Each 512*512 RBG frame is encoded seperately into a 4*64*64 latent. Diffusion runs entirely in this latent space which means that every activation probe is a function of latents, not pixels.
- **CLIP(frozen, ViT-L/14)** This turns the prompt into 77 tokens each of a 768 dim embedding. This is the route through which the prompt enters the model, through cross attention. It is done for all 16 frames so that the text cannot specify motion frame by frame.

- **U-Net(frozen, about 1.5B)** This is the main denoiser that turns noise into video. It has 4 encodeer blocks that lower the resolution from 64*64 to 8*8 with doubling the cahnnels. a middle block and then 4 decoder blocks that raise the resolution back to 64*64. Skip connections are present that pass encoder features to the decoder. The low resolution blocks are where the layout and trajectory can be seen and at high resolution the appearance and texture. The skip connections explain why outermost decoder block can be a strong patch site beacuse it combines encoder detail with the decoded results.
- **ResNet blocks and timestep embedding** Each ResNet layer applies GroupNorm, SiLU and 3*3 convolutions at each frame with a residual connection. The denoising step enters as a sinusoidal emdedding passed through a MLP and added to the features after the first convolution. This is the reson why onel ayer can do differnt jobs at different steps and why our map needs to run over both layers and steps and not just layers.
- **Spatial Transformer** Each has a self attenition layer where each position is given attention wrt other in the frame and a cross attention layer where image positions attend to text token. and a FFN with residual connections. Frames are bacthed so these cannot see other frames. Self attention keeps appearance consistent within a frame and that is used for appearance transfer. Cross attention is where we hook our content coordinates and where the DAAM maps come from .
- **Motion Module(21 modules about 417M)** Inserted after the spatial layers of each block. It reshapes the activation from $(B \cdot F, C, H, W)$ to $(B \cdot H \cdot W, F, C)$ so that the frames form a sequence. It also adds a sinusoidal position embedding and applies two temporal self attention layers and FFN layer and finally adds the result back to the input through a zero initialised output layer. Temporal attention is where the information crosses through the frames hence the only place where motionca can be computed. Without positional encoding forward and backward motion would seem the same . residual addition means that the content and motion mix together and that is why we cannot copy the whole output fo layer. The zero initialisation means training starts from the frozen image model and the motion module learns to add motion on top of it. The attention probabilites are frame to frame matrices what is used for our attention injection baseline.
- **DDIM Sampler** This is the deterministic sampler that makes the same seed run the same clip. It is important for our controlled pairs and for patching to work. Under CFG (scale 7.5) every U-Net call processes an unconditional and a conditional copy; our patches act on both copies by default.

This architecture is advantageouos to us because it has clear seperation in the layers so each and can be hooked and patched. Also it is text conditioned so we can make controlled pairs. On top of that its image weights are frozen so taht we can only learn motion so disentangling is easier. The cost is that some of the seperation is buiilt into the training and we need to probe other models too.

4.2 Work done before the prep-presentation review

Choice of model. We compared three open models. AnimateDiff has an explicit spatial/temporal split in its module tree, mature diffusers support, and fit on a T4 (10.2 GB measured). ModelScope T2V (1.7 B parameters) trains space and time together end to end, which makes it the harder and more interesting case, but the tooling for extracting its attention is thinner. Stable Video Diffusion has the best output quality but is conditioned on an image only, so we could not build matched pairs from one-word prompt changes. We chose AnimateDiff for all deliverables and kept ModelScope as a later robustness check.

Definitions and protocol. From the brief we fixed: content is what an image classifier would see in a single frame (subject identity, colour, texture, layout); motion is how pixels move across frames, regardless of what they belong to. We wrote a roadmap with a register of 22 decisions (D-01 to D-22), among them

the distance (D-01: $1 - \cos$), the static/dynamic split (D-02), how pairs are formed (D-05), the edit metric and success criterion (D-14, D-16), and the baseline (D-17).

Three hypotheses with fixed tests. Each one predicts something that could fail.

- **H1 (layer).** Motion selectivity peaks in a medium-resolution decoder block (`up_blocks.1`), following [35]. Test: the bootstrap argmax over the nine temporal blocks of $M(b) = -\text{mean}_t \, \text{median} \, S_{dyn}(b, t)$ must exclude the extreme-resolution blocks.
- **H2 (timestep).** Motion is settled early in denoising. Test: a single least-squares changepoint τ on the winning block's profile must fall in the first half, with the early segment above the late one.
- **H3 (representation).** The frame mean $\bar{x}$ is content-dominated, and removing it reveals motion selectivity that the raw activation hides. Test: at least 75% of coordinate-steps have S_{stat} significantly content-selective, and at least one temporal coordinate flips from content-selective in $\bar{x}$ to motion-selective in $\tilde{x}$.

Pilots E1–E3 (Kaggle T4).

- **E1, cost.** 94.1 s and 10.2 GB per clip on a T4 at our settings, so the full 360-clip probing set needs about 9.4 GPU-hours.
- **E2, does a fixed noise tensor isolate a word?** Seven clips from one noise tensor: "a golden retriever running on a beach", three subject swaps and three verb swaps. Subject swaps moved CLIP appearance 2.6× more than verb swaps, so the content side of the control works. The motion side failed. A frame-difference motion proxy moved *more* under subject swaps (0.461) than under verb swaps (0.390), because frame differences sit on the silhouette of the object and a new object has a new silhouette (Figure 5). Pixel motion is confounded by content. This is why selectivity is measured on the frame-mean-removed residual, and why content pairs only swap subjects within a body-plan family (a dog for a wolf, not a dog for a person).
- **E3, can we get the activations out?** Hooks on all 95 modules cost 0.6% of generation time if they only record metadata. A full fp16 dump of every coordinate at every step is 87.9 GiB per clip; a summary (channel statistics, per-frame residual norms, one 16×16 map per attention layer) is 44.6 MiB, 2,017× smaller. Storing full activations for 360 clips was therefore impossible, which forced the online pair scoring described next.

4.3 Work done after the prep-presentation review

A few things changed after the prep-presentation review. The pilots showed that storing activations was impossible which lead to the lockstep sampler and online scoring. Second the probing sweep falsified H1 and H3 so teh map did not give a clean motion layer to patch to. Third the predicted component patches did not meet the criterion on test pairs. That lead us to review the literature again to find attention level operators and use H2 timings directly through prompt switching. We then repeated the probing on 3 other models adn added a two directional causal tests to check that the map is not model specific.

Lockstep sampler and online scoring. A selectivity map needs, for every pair, the distance between two clips' activations at every coordinate and step. Caching the anchor clip's activations at all 25 steps for 16 coordinates takes about 7.6 GB of host memory per anchor, and a group has 21 anchors. We instead denoise all 60 clips of a (seed, scene) group *in lockstep*: at step t every anchor runs and its step-t activations are kept, then every partner runs and is scored against them, then the step-t activations are

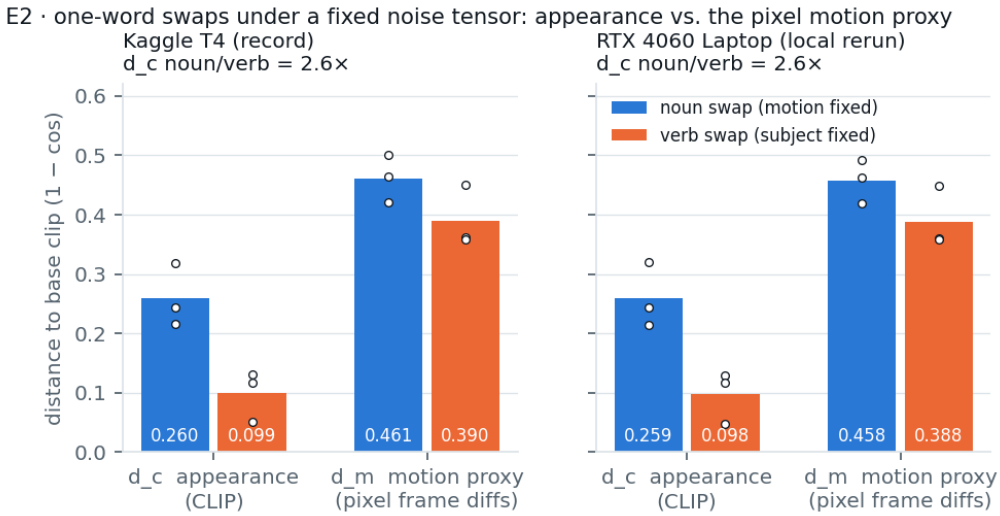


Figure 5: Pilot E2. A pixel-space motion proxy changes more when the subject is swapped (motion held fixed) than when the verb is swapped. Measuring motion in pixel space would have mistaken content for motion.

dropped. Memory holds one step instead of 25, every clip is generated exactly once, and the distances are computed on the full tensors rather than on summaries. The loop re-implements diffusers' `AnimateDiffPipeline` step by step (CFG concatenation, prompt-embedding repetition over frames, the DDIM update). We verified it on the GPU before use: a clip generated alone, inside a group, and with all probes attached gives a final latent identical to `pipe()` (mean absolute difference exactly 0).

Selectivity. For one coordinate (L, t), let d_c be the distance across a content pair (subject swapped) and d_m across a motion pair (verb swapped). Then

$$S = \frac{d_c - d_m}{d_c + d_m} \in [-1, 1], \tag{9}$$

positive when the coordinate tracks what is depicted and negative when it tracks how things move. S_{dyn} uses $\tilde{x}$ and S_{stat} uses $\bar{x}$. Since d_c and d_m come from different pairs, we compute S on matched units: a clip B = (subject s, verb v, scene) that is the end point of both a content pair (family anchor, v) $\to B$ and a motion pair (s, running) $\to B$. This gives 78 units per seed. Each map cell is the median over units with a 1,000-sample bootstrap CI within each seed, averaged over the three seeds.

Linear probes. To check whether motion information is *present* even where selectivity is low, we trained logistic-regression probes for the verb (4 classes) and the subject's family (2 classes) on each coordinate and step. Storing full tensors was impossible, so each clip's $\tilde{x}$ and $\bar{x}$ were compressed with a count sketch to 512 dimensions, a random projection that preserves inner products in expectation. Pipeline: standardise, PCA fitted on the training split only, logistic regression ($C = 1$). We trained on one scene and tested on the other, so a probe cannot succeed by memorising the background.

Component patching. A patch replaces part of the target generation's activation at a coordinate with the source generation's. Besides the usual whole-tensor, channel-subset and α-scaled patches, we added a *component* patch that moves only one term of Eq. (**??**):

$$\text{dynamic:}\quad x_{out} \leftarrow \bar{x}_{tgt} + \tilde{x}_{src}, \qquad\qquad \text{static:}\quad x_{out} \leftarrow \bar{x}_{src} + \tilde{x}_{tgt}. \tag{10}$$

The motion modules write into the residual stream, so a whole-tensor patch there copies content and motion together. The split that defines the map is the natural way to move one and keep the other. In fp16

the obvious implementation $\bar{x}_{out} + x_{src} - \bar{x}_{src}$ is not an exact identity when source and target are the same clip; rounding errors grew over 25 steps to a mean absolute difference of 10^{-2}. We compute it instead as $x_{out} + (\delta - \bar{\delta})$ with $\delta = x_{src} - x_{out}$, which is exactly zero for a self-patch.

Algorithm 1 One edit: source A and N target arms denoised in lockstep from one noise tensor

Require: noise z_T, prompts p_A (source) and p_{tgt}, arms $\{(\text{coords}_k, \text{steps}_k, \text{component}_k, \alpha_k)\}_{k=1}^{N}$
$z^A \leftarrow z_T$; $z^k \leftarrow z_T$ for $k = 1..N$
for t in the 25 DDIM steps **do**
  run U-Net on z^A with p_A; cache A's outputs at every coordinate any arm patches at t
  for $k = 1..N$ **do**
    run U-Net on z^k with p_{tgt}; at each $c \in \text{coords}_k$, if $t \in \text{steps}_k$:
      $\delta \leftarrow x_c^A - x_c^k$; $x_c^k \leftarrow x_c^k + \alpha_k \cdot \text{part}(\delta)$ ▷ part $= \delta - \bar{\delta}$ (dynamic), $\bar{\delta}$ (static) or δ (whole)
    $z^k \leftarrow \text{DDIM}(z^k, \hat{\epsilon}^k, t)$
  $z^A \leftarrow \text{DDIM}(z^A, \hat{\epsilon}^A, t)$; free the step-$t$ cache

Temporal attention injection (baseline). In each temporal attention layer the target uses the source's attention probabilities P_{src} instead of its own over a chosen window of steps, with its own values V. To keep a self-injection exactly null in fp16 we compute $\text{SDPA}(Q, K, V) + \alpha\, W_{out}\big((P_{src} - P)V\big)$, which equals $\big((1-\alpha)P + \alpha P_{src}\big)V$ in exact arithmetic, with P_{src} stored in fp32.

Edit directions and the metric D. Clips A and B differ in subject and verb. In the *motion* direction the target prompt is B's subject with A's verb and we patch $\tilde{x}$, so the output should look like B and move like A. In the *content* direction the target prompt is A's subject with B's verb and we patch $\bar{x}$. We score content fidelity s_c as the mean per-frame CLIP image cosine [27] and motion fidelity s_m as the cosine of dense Farnebäck optical-flow fields [9] averaged over frame pairs. For the motion direction

$$D = \tfrac{1}{2}\big[\hat{s}_c(\text{out}, B) + \hat{s}_m(\text{out}, A)\big] - \tfrac{1}{2}\big[\hat{s}_c(\text{out}, A) + \hat{s}_m(\text{out}, B)\big], \tag{11}$$

with A and B exchanged for the content direction, and $\hat{s}$ min-max normalised over all values in a run. D rewards taking the right property from each clip and penalises taking the wrong one. The quantity we report is $\Delta D = D(\text{arm}) - D(\text{prompt only})$ on the same pair: what the patch adds over simply changing the prompt. The pre-registered success criterion (D-16) has three parts: in both directions, the median ΔD is positive, its 95% paired-bootstrap CI over pairs excludes zero, and subject consistency stays within 10% of the source clip's.

Attention-level edits (Stage 6b). The component patches write into the residual stream and that stream carries both content and motion. H3 already showed us that even the frame mean $\bar{x}$ of a motion module carries some motion, so a static patch also drags some motion along with it. The editing methods in the literature work one level lower, inside the attention. They only change what each position looks at, and the target keeps its own queries, so it keeps its own layout. We built three such operators and tested them with the same pairs, metric and success criterion as before. All the operators and the rule for picking one of them were written down before any of them ran (D-37). Since the test pairs had already been used once, we treat this stage as exploratory and report it separately from the pre-registered verdict.

- **Key/value injection** into spatial self-attention, following MasaCtrl [6] and the content branch of UniEdit [2]. From a chosen step onwards the target's spatial self-attention uses its own queries with the source's keys and values (Eq. (7)). The target gets the source's appearance but keeps its own layout and motion. We have a frame-wise version, where frame f reads source frame f, and a frame-0 version, where every frame reads source frame 0. The frame-0 version is there because frame-wise keys and values also carry the layout of each source frame, and with it some of the source's motion.
- **Temporal attention injection over all 25 steps**, the motion branch of UniEdit. It is the same operator as our baseline (Eq. (8)) but applied over the whole trajectory instead of the first third.

- **Dual-source edits**, where appearance comes from one clip and motion from the other at the same time. In the motion direction the output takes A's temporal attention and B's spatial keys and values. In the content direction it takes A's keys and values and B's temporal attention.

For placement we first used what the papers use: keys and values in `up_blocks.2`–`3` or in every decoder block, starting from step 2 or step 5, and temporal attention in all modules, in `up_blocks.0`, or in `up_blocks.1`, the block that MotionClone [24] and MOFT [35] use. We then added three *probe-placed* arms whose placement comes from our own map instead of a paper: temporal attention in `up_blocks.0` and the mid block (the blocks that lean towards motion under H1) at steps 0–2 (the H2 window) and at all steps, and keys and values in `down_blocks.2`, which has the strongest content selectivity in S_{stat}. That makes 5 motion arms and 6 content arms, run on the 4 dev pairs. The rule picks, for each direction and each family (single-source or dual), the arm with the highest median ΔD among the arms whose subject-consistency ratio is at least 0.9, with ties going to fewer modules. It picked:

- motion, single: temporal attention in every module at every step;
- motion, dual: the same, plus B's keys and values in `up_blocks.2`–`3` from step 2;
- content, single: the probe-placed keys and values in `down_blocks.2`;
- content, dual: A's keys and values with B's temporal attention at every step.

As with the patches, injecting a clip's own keys and values into itself gives back exactly the same clip.

One thing has to be said about the dual-source arms. They read clip B and are then scored partly against B ($s_c(\text{out}, B)$ in the motion direction and $s_m(\text{out}, B)$ in the content direction), so part of their score comes from the way they are built. They answer a different question, which is whether we can swap motion and appearance between two clips, and not whether we can edit a clip with a new prompt. So we report them as a separate family and do not compare them with the prompt-only edit as if they were the same thing.

Prompt switching timed by H2 (Stage 6c). H2 says that motion is decided in the first two of the 25 steps. If that is true, an edit should not need any surgery inside the network at all. We can denoise with one prompt for the first τ steps and then switch to the other prompt. In the motion direction we start with A's prompt and switch to (B's subject, A's verb) at step τ. In the content direction we start with (A's subject, B's verb) and switch to A's prompt. Every track starts from the pair's shared noise. We also built a *masked* switch for the motion direction, following EIDT-V [19]. From step τ on the latent becomes $z \leftarrow M \cdot z + (1 - M) \cdot z_A$, where M is A's subject mask, so everything outside the subject stays as it is in A. The mask comes from A's cross-attention, aggregated as in DAAM [31] but keeping the frame axis: the attention on the subject's tokens is averaged over heads, upsampled, summed over the 16 spatial cross-attention layers and all 25 steps, thresholded at 0.4 of each frame's maximum and dilated by 7 latent pixels. The pre-registered arms use $\tau = 2$, the H2 changepoint. That number came from the probing sweep, so nothing was chosen on the edit pairs. The other values ($\tau \in \{1, 3, 5, 8\}$ and masked $\tau = 3$) are exploratory and show how the result changes with the switch time. If the H2 timing is causal, this curve should bend sharply near $\tau = 2$. So this stage is a test of H2 and an editing method at the same time (D-38).

Motion LoRAs (the trained model). LoRA [17] adds a trainable low-rank update $\Delta W = \frac{\alpha}{r} BA$ to frozen weight matrices. We put LoRA on the query, key, value and output projections of the temporal attention layers, in three placements with the same rank, data and number of steps:

- `full`: all 21 motion modules (9.67 M trainable parameters);
- `probe`: only `up_blocks.0`, the H1 bootstrap set (1.97 M);

- `anti`: `down_blocks.2` and `.3`, the two blocks with the lowest $M(b)$ (2.62 M, more than `probe`), a wrong-place control.

Without the control, "`probe` is as good as `full`" could mean that any block would do. The loss is the standard ϵ-prediction objective of the frozen model,

$$\mathcal{L} = \mathbb{E}_{z_0,\epsilon,t} \left\| \epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t}\, z_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon,\ t,\ c) \right\|_2^2, \tag{12}$$

on 16-frame VAE latents z_0 of real clips with caption c = "a person {verb}"; only the LoRA matrices receive gradients.

We chose LoRA over fine-tuning the whole motion adapter because it trains about 2–10 M parameters instead of 417 M, so it fits on the 8 GB laptop GPU and we can control exactly where it goes. We put it only on the temporal attention projections because that is where information moves across frames, which is also the choice made in the temporal path of MotionDirector [38]. The `anti` LoRA is a bit bigger than `probe`, so if `probe` wins it cannot be because it has more parameters. Diffusers' gradient checkpointing covers only the ResNets of this model, so we wrapped every motion module in checkpointing ourselves to fit training in memory (Section 6).

Upload path: DDIM inversion. Every edit needs a source track whose activations we can cache at every step. An uploaded video has no generation behind it, so we make one. The video is cut to its longest shot, where a cut is a jump in mean grey-level difference between frames that is larger than both 4× the clip's median jump and 20. The shot is resampled to 16 frames, encoded with the VAE and inverted with deterministic DDIM ($\eta = 0$, text-conditional only, no CFG) onto the same 25 timesteps the sampler uses. Each inversion step goes from a cleaner latent z_k to a noisier one,

$$\hat{z}_0 = \frac{z_k - \sqrt{1-\bar{\alpha}_k}\,\epsilon}{\sqrt{\bar{\alpha}_k}}, \qquad z_{k+1} = \sqrt{\bar{\alpha}_{k+1}}\,\hat{z}_0 + \sqrt{1-\bar{\alpha}_{k+1}}\,\epsilon, \tag{13}$$

with ϵ predicted at z_k. During the edit the source track is driven along this trajectory: at each step the U-Net sees the inverted latent, so the activations we cache are the video's, and its own update is replaced by the next inverted latent. The new clip does not start from the inverted noise itself, because inverted latents are not Gaussian and sampling from them with CFG 7.5 gives blotchy, grid-patterned clips even without any edit. It starts from the inverted noise and the seed's Gaussian noise added at equal variance and rescaled to unit variance. Section 7.5 compares this start with the other two.

Experiment 2: replication on other models and causal tests. Experiment 1 had three weak points. First, motion was just a word. The only motion label we had was the verb in the prompt, and the frozen SD1.5 base morphs from frame to frame, so $\tilde{x}$ holds appearance drift as well as real movement. Every layer after cross-attention also sees the verb token, so a verb probe can succeed without any motion being represented. Second, our grid was coarse. We probed one motion module per block, while `up_blocks.1`, the motion site in MOFT, MotionClone and DMT [36], has three. Third, our causal tests went in one direction only and used 12 pairs. Patching in one direction tests whether a site is sufficient but not whether it is necessary [11], and from the width of our CIs we would need about 25 pairs per claim to detect an effect of 0.10. Experiment 2 was built to fix each of these, and all of its rules were fixed before any of its data were generated (D-35).

Four models. We crossed the image weights, SD1.5 or epiCRealism [41] (a community fine-tune of SD1.5), with the motion weights, adapter v1-5-2 or v1-5-3 [40]. Only these weights change. The sampler, the prompts and the CPU-drawn noise are the same in all four cells, so every comparison between cells is paired by prompt and noise. Swapping the base tells us whether the motion site depends on the image features that go into the motion modules, and swapping the adapter tells us whether it belongs to the motion prior. The v1-5-3 adapter has no motion module in the mid block (20 instead of 21), so we match coordinates

by block and position. Each cell has 240 clips: the probing vocabulary with seeds 42 and 1337 and both scenes. Seed 2024 was kept aside for the causal tests.

A finer sweep and measured motion. The sweep hooks 117 coordinates, the 95 from Experiment 1 plus 22 ResNet outputs, because Basu et al. [3] found that much of Stable Diffusion's knowledge sits in the ResNets and self-attention rather than in cross-attention. Every coordinate is count-sketched to 512 dimensions, and $\tilde{x}$ in the refined `up_blocks.0` and `up_blocks.1` coordinates is sketched to 2,048 dimensions at steps 0–14, since sketched cosines close to zero are noisy at 512. Distances between pairs are still exact and computed online in lockstep. To turn motion into something we measure, we ran RAFT optical flow [32] on every clip, removed camera motion by subtracting the median background flow, and used the DAAM subject masks to keep only the subject's flow. CLIP frame embeddings are the content measure.

Quality gate. Before choosing one cell for the expensive steps we scored all 960 clips with checks from VBench [18]: subject consistency (DINO [7] similarity of each frame to the first and to the previous frame), dynamic degree (a clip moves if enough frame pairs have a large top-5% RAFT flow), CLIP-T for text alignment, and X-CLIP [26] to check that the verb is recognisable. The rule was to take the highest subject consistency among the cells with a dynamic share of at least 0.5 and CLIP-T within 0.01 of the best. No cell passed both. Both epiCRealism cells hardly move (dynamic share 0.021 and 0.004), and both SD1.5 cells missed the CLIP-T tolerance by 0.001–0.002. The rule did not say what to do in that case, so before anything else ran we dropped the CLIP-T condition and kept the motion one (D-39), because subject consistency on its own rewards videos that do not move. The primary cell is SD1.5 with v1-5-2, the same model as Experiment 1, and the ladder verbs are walking and running, the two that X-CLIP recognised best.

Looking for presence. On each cell we ran the following.

- **A confirmatory test** of Experiment 1's motion site (`up_blocks.0` motion module 2, steps 0–2, S_{dyn}): each subject's mean selectivity, and an exact one-sided sign-flip test over the 15 subjects (2^{15} patterns). The site replicates if $p < 0.05$. This is the only confirmatory test; everything else here is exploratory.
- **Selectivity maps** with a sign-flip t in every cell and max-$|t|$ family-wise correction over the whole map.
- **ANOVA variance fractions.** The design is balanced, so the variance of an activation splits exactly into subject, verb, scene, seed and subject×verb fractions. This gives a direct "how much is content and how much is motion" number for each layer.
- **RSA with variance partitioning** [22]. For each layer and step, the clip-by-clip distance matrix is regressed on three model matrices: content (CLIP image embeddings), motion (subject-masked flow) and the prompt's CLIP text embedding. The *unique* motion share is the part that flow explains beyond content and text. Significance comes from permuting clips within a seed (49 draws) with max-over-map correction.
- **Debiased CKA** [21] to compare layers within a cell and across cells. With 60–120 clips and 512 features, ordinary CKA is high even for random matrices, so we use the unbiased version.
- **Flow decoding gain.** Ridge regression decodes the part of the flow that changes *within* a prompt (across seeds) from the layer's $\tilde{x}$, and separately from the step-t latent. The gain $G = \max(R^2_{layer}, 0) - \max(R^2_{latent}, 0)$ is the motion information the layer adds over the noisy latent. Variation within a prompt is the one kind of motion that the prompt itself cannot explain.
- **Channel selectivity**, in the style of MOFT, to see whether motion sits in a few channels.

After analysing the first cell, and before selecting anything, we found and corrected three mistakes in the analysis (D-36). The ridge strength had been chosen by leave-one-clip-out CV, which leaks through the paired residuals of a prompt's two seeds, so we replaced it with grouped 4-fold CV over subjects. The RSA was missing the text-embedding matrix that the design asked for; without it the motion share was significant in 82% of cells, because the verb word reaches every layer through the text. And the channel energies stored in fp16 underflowed, so we recomputed them in fp64.

Testing use. The pre-registered rule picked four candidate sites on the primary cell: (1) Experiment 1's site, `up_blocks.0` motion module 2 at steps 0–2, on $\tilde{x}$; (2) the refined coordinate with the highest unique RSA motion share, in a 3-step window at its peak, on $\tilde{x}$; (3) all three `up_blocks.1` motion modules at steps 0–7, on $\tilde{x}$, the site from the literature; and (4) the coordinate with the highest unique RSA content share, at its peak, on $\bar{x}$. Each was tested on 25 motion and 25 content pairs drawn from the held-out seed 2024, by patching in three ways:

- A→B, running B with A's activation, which tests whether the site is *sufficient*;
- B→A, running A with B's activation, which tests whether it is *necessary*;
- C→B, patching from an unrelated clip C, which separates "carries A's property" from "any disruption at all".

Following Zhang and Nanda [37] we normalise the effect,

$$\mathrm{PE}_P = \frac{D_P(x_{patch}) - D_P(x_B)}{D_P(x_A) - D_P(x_B)}, \tag{14}$$

where D_P is a contrastive score for property P (for motion, flow similarity to A minus flow similarity to B). PE is 0 when nothing transfers and 1 when everything does, and the necessity effect NE is defined the same way for B→A. The CIs come from a bootstrap over subjects.

As a second test of necessity we used LEACE [4], a closed-form eraser that makes a concept linearly unpredictable while changing the activation as little as possible. We fitted it on a candidate's $\tilde{x}$ with the verb as the concept (rank at most 3) and applied it as a hook during sampling. The control is a random erasure of the same rank, as in amnesic probing [8]: if LEACE hurts the motion more than the random erasure does, the directions it removed were being used. We re-centred the eraser over frames so that it does not shift $\bar{x}$.

To put all of this together we wrote a decision rule, combining ideas from Hewitt and Liang [13], amnesic probing and Heimersheim and Nanda; the combination itself is ours.

- **E, presence:** the site's unique RSA share or its decoding gain is above zero.
- **S, sufficiency:** the CI of its A→B effect is above zero and above the C→B control.
- **N, necessity:** the CI of its B→A effect is above zero, or LEACE hurts more than random erasure.
- **D, dissociation:** the site moves its own property while the other property stays within ±0.1.

A site "encodes but does not use" a property when E holds and both S and N fail. It "uses" the property when S or N holds, and we say which one. We would only call a site *the* motion locus if it showed a double dissociation against a content site.

Content ladder. To turn one-word swaps into dose-response curves, the ladder changes the subject in steps: a null step ("a dog" → "one dog"), then colour, coat, breed, species, scene and lighting, for 8 base animals

× 2 verbs × 7 variants × 2 seeds. Each layer's distance is regressed on the measured CLIP content change and the measured flow change, which gives the layer's sensitivity to content with the motion taken out. This run was still going when we wrote this report. The design also had a LoRA test (modules picked by the probes against 19 random module sets with the same number of parameters, scored with the Motion Fidelity Score of DMT), MDL probing [33] and attribution patching. We could not run these within our compute budget.

Interface. The brief asks for an interface where a user can test the method on their own inputs. We built a Gradio app (`scripts/app.py`) over an engine class (`src/vdprobe/demo.py`) that loads AnimateDiff, CLIP and the LoRAs once on the first request. It has four tabs.

- **Live edit.** The user picks clip A (subject, verb), clip B, a scene and a seed, a direction and a method: the selected component patch, a custom patch (any block, window and α), whole-tensor patch, attention injection, or prompt + LoRA. The engine runs A, B, the prompt-only arm and the edit in one lockstep group, and shows the four clips with s_c, s_m and D. A single edit has no pool of its own to normalise against, so D uses the pool of the scored test run, and the app says so. Subjects and verbs are free text.
- **Upload a video.** The user's video is cut to its longest shot (shot boundaries found from frame-difference jumps), resampled to 16 frames, encoded, and DDIM-inverted onto the sampler's 25 timesteps. The inverted trajectory then serves as the motion source for a new subject given in a prompt. The default method and starting noise for this path were chosen by experiment (Section 7.5).
- **Results browser.** Every clip of the dev and test sweeps with its scores and the run summary.
- **Where motion lives.** The S_{dyn} and S_{stat} curves for each coordinate and the H1–H3 verdicts.

Every live edit is saved with its GIFs, scores and the full request, so a result shown in the app can be reproduced.

4.4 Loss functions and objectives

Most of what we do needs no training at all. The probing, the patches and the attention edits change no weights, and the edit score D is only used to evaluate. A loss appears only where we train something, and Table 1 lists them.

5 Data set Details

Generated probing set (main data). The probing data are videos generated by the model itself, from the prompt template "a {subject} {verb} {scene}, high quality, detailed". Vocabulary, frozen on 22 Sep: 15 subjects in two body-plan families (quadrupeds: golden retriever, German shepherd, horse, wolf, tiger, fox, deer, brown bear; humans: young man, young woman, athlete, soldier, dancer, child, elderly man), 4 verbs (running, walking, jumping, spinning), 2 scenes (on a beach, in a forest), 3 seeds (42, 1337, 2024). That is 120 clips per seed and 360 in total, each 16 frames of 512×512 RGB. Within a (seed, scene) group every clip shares the same initial noise tensor, so two clips that differ in one word differ only because of that word. Pairs per seed: 104 content pairs (subject swapped within a family, verb fixed), 90 motion pairs (verb swapped, subject fixed) and 56 cross-family pairs, reported separately as a harder subset. Across the three seeds this gives 750 pairs and 300,000 distance rows (16 coordinates × 25 steps per pair). There is no download and no preprocessing beyond the model's own VAE decoding; the data are regenerated exactly from the configuration files.

What is trained	Loss or objective	Why this one
AnimateDiff itself (pretrained, not by us)	$\mathbb{E}\,\|\epsilon - \epsilon_\theta(z_t, t, c)\|_2^2$, Eq. (2)	Defines what the network computes. It was never told about content or motion, so any split is something it learned on its own.
Motion LoRA	the same ϵ-prediction loss, with gradients only into A and B of $\Delta W = \frac{\alpha}{r} BA$	Keeps the model's own objective, so the LoRA learns motion in the model's terms instead of a new task.
Linear probes	multinomial cross-entropy with an L2 penalty ($C = 1$), on PCA of the count-sketched $\tilde{x}$ or $\bar{x}$	Asks whether the information can be read out linearly, which is the usual probing question.
Flow decoder (Exp. 2)	ridge, $\|y - Xw\|^2 + \lambda\|w\|^2$, with λ chosen by grouped 4-fold CV over subjects	The gain over the latent is the motion information the layer adds; grouped CV avoids the leak fixed in D-36.
LEACE eraser (Exp. 2)	closed form: the smallest change to x after which no linear classifier can predict the verb	Removes exactly the linearly readable verb information, to test whether the model needs it.
Not losses	S, D, ΔD, PE, NE	Used only to measure and to evaluate.

Table 1: Every loss or objective used in the project.

Edit pairs. 12 test pairs and 4 separate dev pairs (`configs/edit_pairs.yaml`), frozen before any edit ran. In each pair A and B differ in both subject and verb; the test pairs have 6 per family, use every verb on both sides, and cycle through scenes and seeds. The dev pairs are used only to choose the patch configuration, so the test pairs never influence it.

HMDB51 clips for the LoRAs. Real human-action videos from HMDB51 [23], fetched from the Hugging Face mirror `divm/hmdb51` [43] at a pinned revision. HMDB51 has about 6,800 clips in 51 action classes, collected from films and web video; the clips we use are 240 pixels high at 30 fps. We use four classes that match our verbs (run, walk, jump, turn → spinning) and filter on HMDB's own metadata tags: full body visible, no camera motion, one person, quality not bad, at most one clip per source film. That leaves 19 clips (run 5, walk 5, jump 4, turn 5), 21 to 168 frames each, 2.0 MB in total, recorded with SHA-256 hashes in `data/hmdb51_lora/manifest.json`. The last clip of each class (sorted by name) is held out for validation: 15 train, 4 validation.

Preprocessing: 16 frames at stride $\min(2, \lfloor (n-1)/15 \rfloor)$ from a random start, a random square crop of full height, resizing to 256×256 with antialiasing, scaling to $[-1, 1]$, and encoding with the frozen VAE. Validation uses a fixed window and a centre crop. While debugging the upload path we found that 11 of the 19 clips contain a hard scene cut (a mean grey-level jump larger than both 4× the clip's median and 20). In the first recipe some training windows spanned a cut. The *cut-free* recipe (D-34) draws every window inside a single shot of at least 16 frames, chosen with probability proportional to its length; a clip without such a shot uses its longest shot with the last frame repeated.

6 Experiments

Hardware and software. A laptop RTX 4060 (8 GB, 15 GB RAM) for development, controls, LoRA training (first recipe), scoring and the interface; Kaggle sessions with 2× Tesla T4 (16 GB each) for the sweeps and the cut-free LoRAs, one job per GPU. Python 3.10, PyTorch 2.10 (2.11 on Kaggle), diffusers 0.40.0 [42], transformers 5.20, PEFT 0.21.2, scikit-learn 1.7.2, OpenCV, Gradio 6.30. Every result file records the software versions, GPU, timings, peak memory, and the resolved model repository and com-

mit hash. One clip costs 65 s on the laptop and 90–97 s on a T4.

Code. https://github.com/rujulaaaa/IE643. The README lists the command for every stage. The suite of 51 CPU tests runs on a tiny random U-Net in about 10 s and covers hooks, patches, controls, the analysis on a synthetic sweep with a planted answer, the metrics and the interface engine.

E4. Lockstep verification. Four checks against `pipe()`: two `pipe()` runs (floor), a solo lockstep clip, a clip inside a three-clip group, and a clip with all probes attached. All four final latents matched exactly. A self-pair gave distances below 3×10^{-7}; a verb swap gave a median d of 0.555 on $\tilde{x}$ and 0.083 on $\bar{x}$.

E5. Probing sweep. Six lockstep groups of 60 clips (3 seeds × 2 scenes), 16 coarse coordinates × 25 steps. The coordinates are one representative per block: the last motion module of each of the 9 blocks, and the cross-attention of the last spatial transformer of each of the 7 blocks that have one (the last module is closest to the block output, and cross-attention is where the prompt enters the spatial path). Kaggle, 2×T4, 5.2 hours. Analysis on the CPU: 1,000 bootstrap resamples, probes in 836 s.

E6. Patching controls on the real model. Null patches (source = target) for every patch type; a misindexed null patch (the source's activation from two steps earlier), which *must* fail; an unpatched arm compared with the same clip generated as another group's source; and a symmetry check, A→B against B→A on `up_blocks.0` over the first third of the steps. The first run of these controls failed two of them. The component null patch gave a mean absolute difference of 10^{-2} (the fp16 identity issue above). Attention recording perturbed the source clip by 6×10^{-3}, because the recorder returned its own explicit-softmax output instead of the stock one. A second run left the null attention injection at 2.3×10^{-3}, which led to the SDPA + $\alpha W_{out}(\cdot)$ form. The third run is the run of record (Table 3).

E7. Dev sweep (configuration selection). 4 dev pairs, 51 arms each: component patches on the motion-module outputs of each of the 9 temporal blocks over two windows (first third of the steps, i.e. steps 0–8; all 25), on all 21 motion modules over four windows (one step, five steps, first third, all), both directions, plus whole-tensor and attention-injection references. Rule (D-31, fixed before scoring): per direction, the component arm with the largest median ΔD among those whose median subject-consistency ratio is ≥ 0.9, ties going to fewer modules. Kaggle, 2.6 h. Selected: motion, $\tilde{x}$ patch on `up_blocks.3` motion modules, steps 0–8 (dev median ΔD +0.080); content, $\bar{x}$ patch on `down_blocks.3` motion modules, steps 0–8 (+0.015).

E8. Test sweep. 12 test pairs, 18 clips each: source, B, and per direction the prompt-only arm, the selected patch, the selected patch at $\alpha \in \{0.25, 0.5, 0.75\}$, the whole-tensor patch at the same place; temporal attention injection over steps 0–8 in all 42 temporal attention layers; prompt + each of the three LoRAs. Kaggle, 10,017 s. The first submission failed before generating anything because the Kaggle image's `torchao` was incompatible with the newest PEFT; we pinned PEFT 0.21.2 and removed `torchao`. Scoring (VAE decode, CLIP, optical flow) ran on the laptop in 15 minutes, with 10,000 paired bootstrap resamples per CI.

E9. Motion LoRAs, first recipe. Rank 32, $\alpha = 32$, Gaussian initialisation, AdamW (learning rate 5×10^{-5}, weight decay 10^{-2}), batch 1, 800 steps, timesteps uniform over the 1,000 training steps, fp16 frozen weights with fp32 LoRA weights under autocast, seed 0. The first attempt ran out of memory on the 8 GB GPU at step 1: diffusers' gradient checkpointing covers only the ResNets of this model, not the motion modules where the LoRA sits. We wrapped every motion module in non-reentrant checkpointing (same gradients); peak memory fell to 5.0 GB. 17–22 minutes per variant. Validation: the 4 held-out clips at 8 fixed timesteps × 2 fixed noise tensors, identical for every variant and for the untrained model.

E10. Motion LoRAs, cut-free with three seeds (D-34). The same recipe with cut-free windows, seeds 0, 1 and 2, nine LoRAs, on Kaggle (22–30 minutes each on a T4). On the 12 test pairs we generated one prompt + LoRA arm per LoRA and attention injection over all 25 steps (the best motion arm in the dev sweep, which the selection rule did not allow us to pick), regenerated the source, B and the prompt-

only arm, and scored everything in one pool with E8. Kaggle, 2×T4: about 1.9 h of training and then 1.7–2.0 h of edits per GPU.

E11. Upload path. Interface checks on real HMDB51 clips, comparing three starting points for the new clip (the inverted noise, a unit-variance blend of the inverted and a Gaussian noise, Gaussian noise alone) and three methods (selected $\tilde{x}$ patch, attention injection over the first third, attention injection over all steps).

7 Results

7.1 Where content and motion live

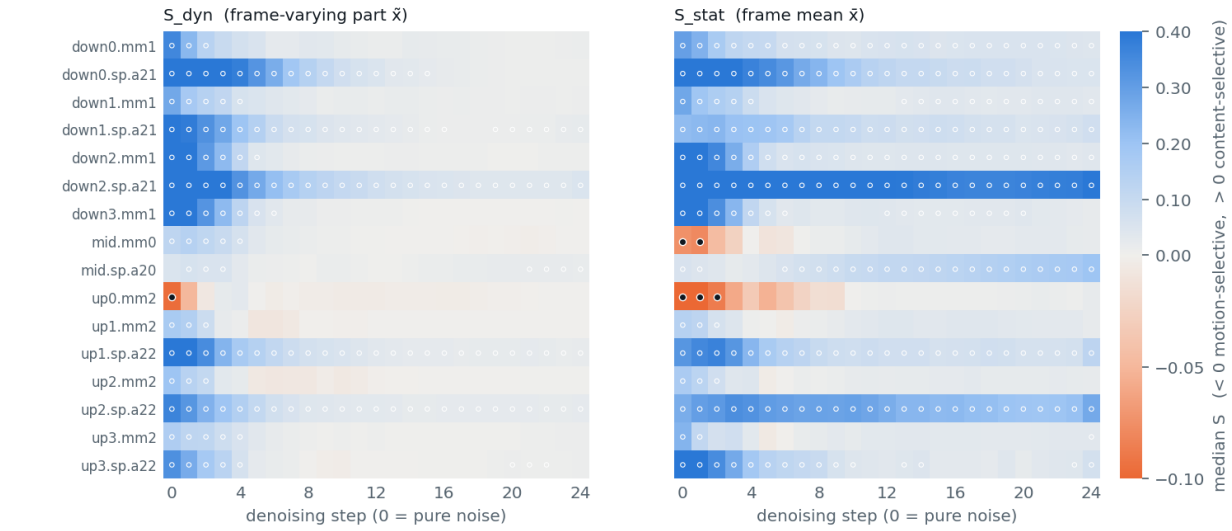


Figure 6: Selectivity maps over 16 coordinates and 25 denoising steps (`mm` = motion module, `sp.a2x` = spatial cross-attention). Left: S_{dyn} on the frame-varying part. Right: S_{stat} on the frame mean. Blue is content-selective, orange motion-selective. Only `up0.mm2` (and `mid.mm0` in $\bar{x}$) is motion-selective, and only in the first steps.

Figure 6 is the central result of the probing stage. Almost every cell is blue: a subject swap moves almost every layer more than a verb swap does, including the temporal layers. The spatial cross-attention layers are the most content-selective. Every map fades towards zero after about step 9, where content and motion swaps move the activations by similar amounts. Table 2 gives the three pre-registered tests.

Two observations go beyond the tests. First, the H3 result is the opposite of what we expected: at the very first steps the cleaner motion signal is in $\bar{x}$, not $\tilde{x}$. S_{stat} has five motion-significant cells (`up0.mm2` at steps 0–2, `mid.mm0` at steps 0–1) and S_{dyn} only one. The static share of the activation energy, $F\|\bar{x}\|^2/\|x\|^2$, is 0.39–0.70 in the motion modules but 0.82–0.9996 in the spatial cross-attention layers, so "the frame mean is content" holds on the spatial path, not the temporal one. Second, on `up0.mm2` the motion profile dips below zero at steps 3–4 (−0.024, −0.030) before settling. Our two-segment test was not designed to detect that, and we have not interpreted it.

Robustness. Cosine and relative-L2 distances give nearly the same map (Spearman 0.987), and the median of per-unit ratios agrees with the ratio of medians (0.930). Seed-to-seed agreement of the S_{dyn} map is

Hypothesis	Verdict	Evidence
H1: motion peaks at `up_blocks.1`	falsified	$M(b)$ is highest at `up_blocks.0` (+0.005), the argmax in 1000/1000 resamples. Every other block is negative: `up1` −0.017, `up2` −0.019, `mid` −0.026, `down0` −0.052, `down2` −0.075. The peak is sharp but at the extreme (8×8) resolution the test excluded.
H2: motion is settled early	survives	On `up0.mm2`, $m(t) = 0.096, 0.050, 0.007$, then about 0. Changepoint $\tau = 2$, 95% CI [1, 3]; early mean 0.073 vs. late −0.001.
H3: removing $\bar{x}$ reveals hidden motion	falsified	$f_{stat} = 0.65 < 0.75$ and no coordinate flips. $S_{dyn} < S_{stat}$ almost everywhere (e.g. `down2` cross-attention 0.43 → 0.17), so the split moves every coordinate towards motion, but never past zero.

Table 2: Pre-registered hypothesis tests (78 matched units per seed, 3 seeds, 1,000 bootstrap resamples).

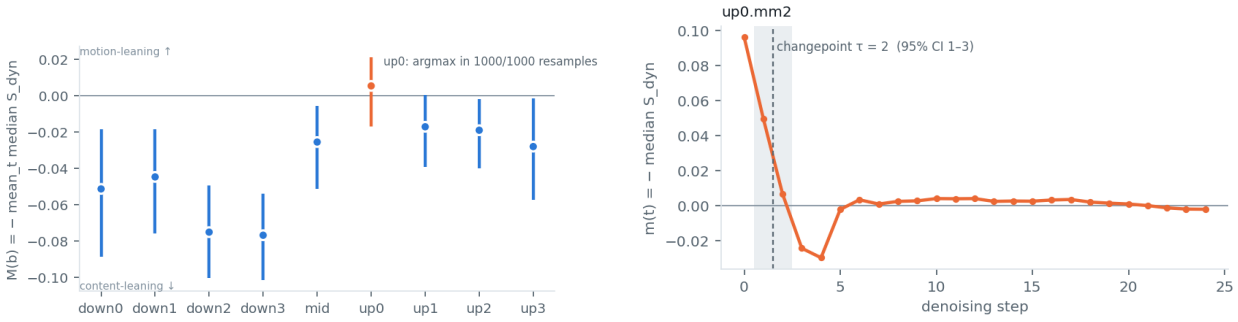


Figure 7: Left: motion selectivity $M(b)$ of the nine temporal blocks with 95% CIs; only `up_blocks.0` is above zero. Right: the motion-selectivity profile of `up0.mm2` over the denoising steps, with the fitted changepoint and its CI.

only moderate (0.58, 0.69, 0.73), so cell-level detail should not be over-read.

Linear probes. With the scene split, the verb is decodable well above chance (0.25) in places, for example 0.74 on the mid-block cross-attention (Figure 8). Much of this is the verb token leaking through the prompt: on that coordinate the probe on $\bar{x}$ stays at 1.0 at every step, because cross-attention reads the text embedding directly. On the temporal modules the frame mean also decodes the verb better than the residual at every step. Selectivity and decodability are different questions: a layer can carry the verb and still respond more to a change of subject.

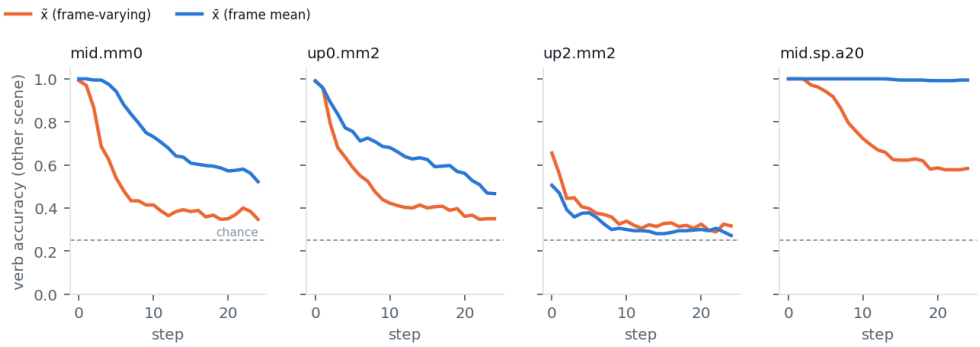


Figure 8: Verb-probe accuracy over the denoising steps on four coordinates, trained on one scene and tested on the other (chance 0.25). $\bar{x}$ decodes the verb at least as well as $\tilde{x}$ everywhere.

7.2 Patching controls

Control	Result	Verdict
Null patch: whole tensor, $\tilde{x}$ only, $\bar{x}$ only, attention injection	MAD 0, PSNR ∞	pass
Mis-indexed null patch (source from 2 steps earlier)	MAD 0.103, PSNR 25.1 dB	detected, as required
Unpatched arm vs. the same clip as another group's source	MAD 0	pass (no leakage)
Symmetry, `up_blocks.0`, steps 0–8: $f(A \to B)$ vs. $f(B \to A)$	0.171 vs. 0.083	fail (ratio $2.06 > 2$)

Table 3: Controls on the real model (run of record). MAD = mean absolute difference of the final latents.

The patching machinery is exact: every null patch reproduces the unpatched clip bit for bit, and a deliberately wrong patch is caught. The symmetry control fails our own criterion. The effect has the same sign both ways, but moving the golden retriever's run into the walking horse transfers twice as much as the reverse. One explanation is that a running clip has more motion to move; we tested only one pair, so this stays a conjecture.

7.3 Edits on the held-out test pairs

Direction	Arm	median D	median ΔD	$\Delta D > 0$	95% CI
keep A's motion	prompt only	0.149	0	–	–
	$\tilde{x}$ patch, `up_blocks.3`, steps 0–8 (selected)	0.195	+0.041	8/12	[−0.027, 0.143]
	$\tilde{x}$ patch at $\alpha = 0.5$	0.236	+0.038	9/12	includes 0
	whole-tensor patch, same place	0.074	−0.036	5/12	[−0.109, 0.047]
	attention injection, steps 0–8 (baseline)	0.269	**+0.105**	10/12	**[0.001, 0.204]**
keep A's content	prompt only	0.090	0	–	–
	$\bar{x}$ patch, `down_blocks.3`, steps 0–8 (selected)	0.198	+0.068	10/12	[−0.010, 0.158]
	$\bar{x}$ patch at $\alpha = 0.5$	0.199	+0.029	9/12	[0.001, 0.152]
	whole-tensor patch, same place	0.133	−0.026	4/12	[−0.080, 0.106]

Table 4: Test-sweep results (12 pairs). ΔD is the paired gain over the prompt-only arm; the CI is a 95% paired bootstrap of the difference in medians (10,000 resamples). Subject consistency stayed within 1.2% of the source for every patch arm.

Table 4 answers the editing question. The pre-registered criterion is **not met**: the selected component patches beat the prompt-only edit on the median in both directions, in 8/12 and 10/12 pairs, with consistency intact, but neither CI excludes zero. Three points are still clear.

- Component patches beat whole-tensor patches at the same site in both directions. The whole-tensor patch copies A wholesale: in the motion direction it reaches $s_m(\text{out}, A) = 0.82$, but its content also moves to A ($s_c(\text{out}, A) = 0.91$), so $\Delta D < 0$. The selected $\tilde{x}$ patch raises $s_m(\text{out}, A)$ from 0.23 to 0.67 while $s_c(\text{out}, B)$ drops only from 0.85 to 0.81. This is the entanglement the split was meant to remove, and it removes much of it.
- Where motion is *represented* and where patching *uses* it are different places. The map peaks at `up_blocks.0`; the best patch site in the dev sweep was the outermost, 64×64 decoder block `up_blocks.3`, where patching `up_blocks.0` gave only +0.030.
- Temporal attention injection, which needs no map at all, is the strongest motion edit and the only arm whose CI excludes zero, and only by 0.001.

Attention injection over all 25 steps (E10). In the dev sweep the best motion arm overall was attention injection over the whole trajectory (ΔD +0.151), which the selection rule excluded because it is not a component patch. We ran it on the test pairs in the LoRA run. That run regenerated the source, B and the prompt-only clip of every pair, and all three came out bit-identical to the test sweep's (mean absolute latent difference 0 on all 12 pairs), so the two runs can be scored in one pool. Over all steps, injection reaches median $\Delta D = +0.120$, positive on 11 of 12 pairs, with CI [0.020, 0.203]: the clearest positive result of the project. It raises $s_m(\text{out}, A)$ to 0.76 while $s_c(\text{out}, B)$ stays at 0.81. Adding the new arms changes the min-max pool, so the other arms move slightly (the selected $\tilde{x}$ patch +0.045, CI [−0.019, 0.148]; injection over steps 0–8 +0.108, CI [0.006, 0.202]); Table 4 keeps the numbers of record from the test run alone.



Figure 9: Motion direction on test pair 1 ("a golden retriever running" → "a horse walking", beach, seed 42). Target prompt: "a horse running". The prompt-only horse runs, but in its own way; with the $\tilde{x}$ patch it picks up the dog's splash and stride, and its flow similarity to A rises from 0.38 to 0.60. D falls slightly on this pair because some of A's appearance comes along (the pale mane and spray).

Figures 9 and 10 show one pair in both directions. They also show the limits of our metrics. CLIP similarity is a coarse measure of identity, and the flow cosine rewards matching *where* pixels move, which includes



Figure 10: Content direction on the same pair. Target prompt: "a golden retriever walking". The $\bar{x}$ patch on `down_blocks.3` keeps the source dog's build, coat and raised tail more closely than the prompt alone (s_c to A 0.87 $\rightarrow$ 0.91) while it walks.

splashes and camera-like drift as well as the animal's legs.

7.4 Motion LoRAs

Variant	Params	Held-out loss at step 800 first recipe	Held-out loss at step 800 cut-free, seeds 0/1/2	Test pairs, prompt + LoRA, median ΔD first recipe	Test pairs, prompt + LoRA, median ΔD cut-free, per seed
untrained	–	0.0996	0.0982	–	–
`full`	9.67 M	0.0638	0.0627 / 0.0627 / 0.0627	−0.021	+0.004 / +0.008 / −0.013
`probe`	1.97 M	0.0652	0.0641 / 0.0641 / 0.0641	+0.023	+0.014 / +0.015 / +0.009
`anti`	2.62 M	0.0646	0.0635 / 0.0636 / 0.0635	−0.020	−0.011 / −0.031 / −0.001

Table 5: Motion LoRAs. The untrained loss differs between recipes because the cut-free validation window is the centre of the longest shot. Cut-free ΔD is scored in one pool with the test run (E10). Every ΔD CI in the table includes zero.

On held-out loss, placement matters less than size. In both recipes `full` is lowest and the wrong-place control `anti` ends *below* the map-guided `probe`, with 33% more parameters. The ordering is identical for all three cut-free seeds, and the spread between seeds is below 10^{-4}, so it is not noise. The validation loss is not specific to motion, though (Figure 11). The edit test is the better measure, and there the first-recipe

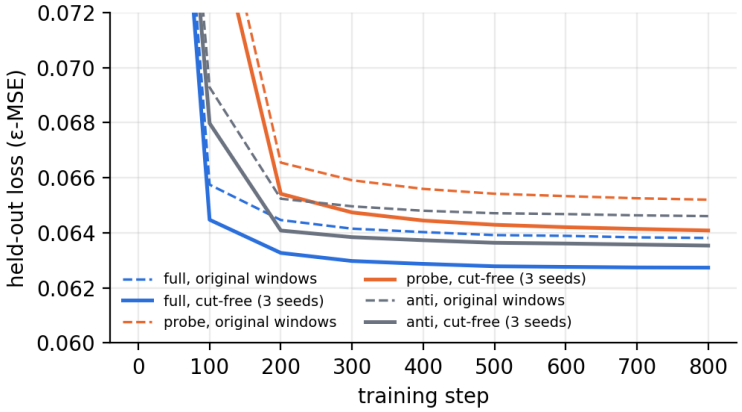


Figure 11: Held-out loss during training. Most of the drop happens in the first 100 steps for every variant, which looks more like adapting to HMDB51's look (grain, low resolution) than to its motion.

`probe` LoRA had the only positive median ΔD of the three. The cut-free retrain repeats this: `probe` is positive in all three seeds (+0.009 to +0.015, 6–7 of 12 pairs), while `full` and `anti` scatter around zero. So on the edit test the map-guided placement does come out ahead, consistently but by a small margin, and no single CI excludes zero. The raw scores qualify the gain. With `probe`, $s_m(\text{out}, A)$ *falls* from 0.23 to 0.14–0.17 while $s_m(\text{out}, B)$ falls further, from 0.17 to 0.06–0.08. The LoRA makes the clip move less like B rather than more like A. That is what we should expect: it learned generic HMDB51 running, walking, jumping and turning, not clip A's particular motion, so we compare the three LoRAs with one another and not with the patches.

What the full LoRA learns, block by block. For the `full` LoRA we computed the relative update $\|\Delta W\|_F / \|W\|_F$ per temporal block and ranked the nine blocks by it, by selectivity $M(b)$, by verb-probe accuracy on $\tilde{x}$ and by usefulness for patching (dev ΔD). The LoRA's largest updates follow the probe ranking (Spearman +0.83, $p = 0.005$), peaking at `up_blocks.0` where both the map and the probe peak; they do not follow patching usefulness (−0.27). The three low-resolution blocks have larger relative updates in general, so part of this agreement may be resolution rather than motion; we have not separated the two.

7.5 Upload path

New clip starts from	prompt only	selected $\tilde{x}$ patch	attention, steps 0–8	attention, all steps
inverted noise	0.118	0.519 (hybrid, artefacts)	0.309	**0.668**
blend of inverted and Gaussian	0.322	0.025 (diverges)	0.395	**0.656**
Gaussian noise	0.085	0.087 (diverges)	−0.053	0.366

Table 6: Uploaded video (a baseball pitch) turned into "a tiger jumping": flow cosine of the new clip to the upload. CLIP match to the target prompt is 0.33–0.36 for every clip that did not diverge.

The component patch does not survive this setting (Table 6). Logging norms inside the patcher showed why. For a generated pair the frame-varying parts of source and target already agree closely, since both start from the same noise, so the patch is a small nudge. For an uploaded video they are unrelated: the difference at `up_blocks.3` is up to 12,400 against an output norm of 8–21 thousand at step 0, the target's norm grows every step, and the final latent has a standard deviation of 26 instead of 0.7. The patch is a

tool for pairs that share their initial noise, not a layout-free motion transplant. The interface therefore defaults to attention injection over all steps from the blended start. A second clip exposed another limit: a woman walking towards the camera became a scene that grows towards the camera, while the robot in it stayed small and nearly still. Temporal attention carries *where* things move in the frame, not *which object* moves.

7.6 Interface

All four tabs were checked through the running app with the same handlers as the buttons. A live edit (4 clips) takes 4.5–7 minutes on the laptop GPU at 6.4 GB peak; an upload edit about 4.5 minutes; the results browser and the maps need no GPU. Invalid combinations (attention injection in the content direction) are rejected with a message rather than an error.

8 Plan for Novelty Assessment

The results leave four open questions, and the plan follows them.

1. **Is the split an artefact of AnimateDiff's training recipe?** Its spatial layers were frozen while the motion module was trained, so content *had* to stay in the spatial path. We will run the same sweep (same vocabulary, same tests) on a model trained jointly on space and time, such as ModelScope T2V or VideoCrafter2. If H1–H3 come out the same, the map describes video diffusion; if not, it describes AnimateDiff.
2. **Combine what each method is good at.** Attention injection transfers *where* things move, and the $\tilde{x}$ patch carries *which* motion at the cost of some appearance. A combined arm, attention injection with a mask from the subject's cross-attention so that motion lands on the object, is the most direct fix for the failure in the upload path, and can be tested on the same 12 pairs.
3. **Statistical power.** Several arms have 9–10 of 12 positive pairs and CIs that miss zero by less than 0.03. We will add a second set of 24 pairs drawn by the same rule and report both sets, and run the symmetry control on all of them instead of one pair.
4. **Better measurements.** Replace optical-flow cosine with point tracks (CoTracker [20]) and CLIP with DINO features for identity, and separate the resolution effect in the LoRA update ranking by normalising per resolution level.

9 Conclusion

We asked where a video diffusion U-Net keeps content and where it keeps motion, using AnimateDiff and a controlled set of 750 one-word-swap pairs generated from shared noise. The answer is narrower than the architecture suggests. Content dominates almost every layer, temporal ones included. Motion is selective in only one place, the motion module just after the bottleneck, and only during the first two or three of 25 denoising steps. Removing the frame mean shifts every layer towards motion but reveals no hidden motion layer, and two of our three pre-registered hypotheses were falsified. Turning the same static/dynamic split into a patch gives edits that beat changing the prompt and beat whole-tensor patching in both directions, but not by a margin that 12 test pairs can confirm. Temporal attention injection did better, and over all 25 steps it is the one edit whose gain is clearly above zero. Where the map says motion is represented is not where patching works best. A LoRA placed by the map was not better on held-out loss than

one placed where the map says motion is absent, but it was the only placement that improved the edit in all three seeds, by a small margin. The measurements are exact (null patches are bit-identical) and every decision was fixed before the data it affected, so the negative results are results. All code, the LoRAs and an interface for trying the edits are released with this report.

References

[1] Vatsal Baherwani, Yixuan Ren, and Abhinav Shrivastava. Characterizing motion encoding in video diffusion timesteps, arXiv preprint, 2025. https://arxiv.org/abs/2512.22175

[2] Jianhong Bai, Tianyu He, Yuchi Wang, Junliang Guo, Haoji Hu, Zuozhu Liu, and Jiang Bian. UniEdit: A unified tuning-free framework for video motion and appearance editing, arXiv preprint, 2024. https://arxiv.org/abs/2402.13185

[3] Samyadeep Basu, Nanxuan Zhao, Vlad Morariu, Soheil Feizi, and Varun Manjunatha. Localizing and editing knowledge in text-to-image generative models, International Conference on Learning Representations (ICLR), 2024. https://arxiv.org/abs/2310.13730

[4] Nora Belrose, David Schneider-Joseph, Shauli Ravfogel, Ryan Cotterell, Edward Raff, and Stella Biderman. LEACE: Perfect linear concept erasure in closed form, Advances in Neural Information Processing Systems (NeurIPS), 2023. https://arxiv.org/abs/2306.03819

[5] Andreas Blattmann, Robin Rombach, Huan Ling, Tim Dockhorn, Seung Wook Kim, Sanja Fidler, and Karsten Kreis. Align your latents: High-resolution video synthesis with latent diffusion models, IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2023. https://arxiv.org/abs/2304.08818

[6] Mingdeng Cao, Xintao Wang, Zhongang Qi, Ying Shan, Xiaohu Qie, and Yinqiang Zheng. MasaCtrl: Tuning-free mutual self-attention control for consistent image synthesis and editing, IEEE/CVF International Conference on Computer Vision (ICCV), 2023. https://arxiv.org/abs/2304.08465

[7] Mathilde Caron, Hugo Touvron, Ishan Misra, Hervé Jégou, Julien Mairal, Piotr Bojanowski, and Armand Joulin. Emerging properties in self-supervised vision transformers, IEEE/CVF International Conference on Computer Vision (ICCV), 2021. https://arxiv.org/abs/2104.14294

[8] Yanai Elazar, Shauli Ravfogel, Alon Jacovi, and Yoav Goldberg. Amnesic probing: Behavioral explanation with amnesic counterfactuals, Transactions of the Association for Computational Linguistics (TACL), 2021. https://aclanthology.org/2021.tacl-1.10/

[9] Gunnar Farnebäck. Two-frame motion estimation based on polynomial expansion, Scandinavian Conference on Image Analysis (SCIA), 2003. https://doi.org/10.1007/3-540-45103-X_50

[10] Yuwei Guo, Ceyuan Yang, Anyi Rao, Zhengyang Liang, Yaohui Wang, Yu Qiao, Maneesh Agrawala, Dahua Lin, and Bo Dai. AnimateDiff: Animate your personalized text-to-image diffusion models without specific tuning, International Conference on Learning Representations (ICLR), 2024. https://arxiv.org/abs/2307.04725

[11] Stefan Heimersheim and Neel Nanda. How to use and interpret activation patching, arXiv preprint, 2024. https://arxiv.org/abs/2404.15255

[12] Amir Hertz, Ron Mokady, Jay Tenenbaum, Kfir Aberman, Yael Pritch, and Daniel Cohen-Or. Prompt-to-prompt image editing with cross attention control, International Conference on Learning Representations (ICLR), 2023. https://arxiv.org/abs/2208.01626

[13] John Hewitt and Percy Liang. Designing and interpreting probes with control tasks, Conference on Empirical Methods in Natural Language Processing (EMNLP), 2019. https://arxiv.org/abs/1909.03368

[14] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models, Advances in Neural Information Processing Systems (NeurIPS), 2020. https://arxiv.org/abs/2006.11239

[15] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance, NeurIPS Workshop on Deep Generative Models and Downstream Applications, 2021. https://arxiv.org/abs/2207.12598

[16] Jonathan Ho, Tim Salimans, Alexey Gritsenko, William Chan, Mohammad Norouzi, and David J. Fleet. Video diffusion models, Advances in Neural Information Processing Systems (NeurIPS), 2022. https://arxiv.org/abs/2204.03458

[17] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and

Weizhu Chen. LoRA: Low-rank adaptation of large language models, International Conference on Learning Representations (ICLR), 2022. https://arxiv.org/abs/2106.09685

[18] Ziqi Huang, Yinan He, Jiashuo Yu, Fan Zhang, Chenyang Si, Yuming Jiang, Yuanhan Zhang, Tianxing Wu, Qingyang Jin, Nattapol Chanpaisit, Yaohui Wang, Xinyuan Chen, Limin Wang, Dahua Lin, Yu Qiao, and Ziwei Liu. VBench: Comprehensive benchmark suite for video generative models, IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2024. https://arxiv.org/abs/2311.17982

[19] Diljeet Jagpal, Xi Chen, and Vinay P. Namboodiri. EIDT-V: Exploiting intersections in diffusion trajectories for model-agnostic, zero-shot, training-free text-to-video generation, IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2025. https://arxiv.org/abs/2504.06861

[20] Nikita Karaev, Ignacio Rocco, Benjamin Graham, Natalia Neverova, Andrea Vedaldi, and Christian Rupprecht. CoTracker: It is better to track together, European Conference on Computer Vision (ECCV), 2024. https://arxiv.org/abs/2307.07635

[21] Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. Similarity of neural network representations revisited, International Conference on Machine Learning (ICML), 2019. https://arxiv.org/abs/1905.00414

[22] Nikolaus Kriegeskorte, Marieke Mur, and Peter Bandettini. Representational similarity analysis: connecting the branches of systems neuroscience, Frontiers in Systems Neuroscience, 2008. https://doi.org/10.3389/neuro.06.004.2008

[23] Hildegard Kuehne, Hueihan Jhuang, Estíbaliz Garrote, Tomaso Poggio, and Thomas Serre. HMDB: A large video database for human motion recognition, IEEE International Conference on Computer Vision (ICCV), 2011. https://doi.org/10.1109/ICCV.2011.6126543

[24] Pengyang Ling, Jiazi Bu, Pan Zhang, Xiaoyi Dong, Yuhang Zang, Tong Wu, Huaian Chen, Jiaqi Wang, and Yi Jin. MotionClone: Training-free motion cloning for controllable video generation, International Conference on Learning Representations (ICLR), 2025. https://arxiv.org/abs/2406.05338

[25] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT, Advances in Neural Information Processing Systems (NeurIPS), 2022. https://arxiv.org/abs/2202.05262

[26] Bolin Ni, Houwen Peng, Minghao Chen, Songyang Zhang, Gaofeng Meng, Jianlong Fu, Shiming Xiang, and Haibin Ling. Expanding language-image pretrained models for general video recognition, European Conference on Computer Vision (ECCV), 2022. https://arxiv.org/abs/2208.02816

[27] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision, International Conference on Machine Learning (ICML), 2021. https://arxiv.org/abs/2103.00020

[28] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models, IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022. https://arxiv.org/abs/2112.10752

[29] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-Net: Convolutional networks for biomedical image segmentation, International Conference on Medical Image Computing and Computer-Assisted Intervention (MICCAI), 2015. https://arxiv.org/abs/1505.04597

[30] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models, International Conference on Learning Representations (ICLR), 2021. https://arxiv.org/abs/2010.02502

[31] Raphael Tang, Linqing Liu, Akshat Pandey, Zhiying Jiang, Gefei Yang, Karun Kumar, Pontus Stenetorp, Jimmy Lin, and Ferhan Ture. What the DAAM: Interpreting Stable Diffusion using cross attention, Annual Meeting of the Association for Computational Linguistics (ACL), 2023. https://aclanthology.org/2023.acl-long.310/

[32] Zachary Teed and Jia Deng. RAFT: Recurrent all-pairs field transforms for optical flow, European Conference on Computer Vision (ECCV), 2020. https://arxiv.org/abs/2003.12039

[33] Elena Voita and Ivan Titov. Information-theoretic probing with minimum description length, Conference on Empirical Methods in Natural Language Processing (EMNLP), 2020. https://arxiv.org/abs/2003.12298

[34] Jay Zhangjie Wu, Yixiao Ge, Xintao Wang, Stan Weixian Lei, Yuchao Gu, Yufei Shi, Wynne Hsu, Ying Shan, Xiaohu Qie, and Mike Zheng Shou. Tune-A-Video: One-shot tuning of image diffusion models for text-to-video generation, IEEE/CVF International Conference on Computer Vision (ICCV), 2023. https://arxiv.org/abs/2212.11565

[35] Zeqi Xiao, Yifan Zhou, Shuai Yang, and Xingang Pan. Video diffusion models are training-free motion interpreter and controller, Advances in Neural Information Processing Systems (NeurIPS), 2024. https://arxiv.org/abs/2405.14864

[36] Danah Yatim, Rafail Fridman, Omer Bar-Tal, Yoni Kasten, and Tali Dekel. Space-time diffusion features for zero-shot text-driven motion transfer, IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2024. https://arxiv.org/abs/2311.17009

[37] Fred Zhang and Neel Nanda. Towards best practices of activation patching in language models: Metrics and methods, International Conference on Learning Representations (ICLR), 2024. https://arxiv.org/abs/2309.16042

[38] Rui Zhao, Yuchao Gu, Jay Zhangjie Wu, David Junhao Zhang, Jiawei Liu, Weijia Wu, Jussi Keppo, and Mike Zheng Shou. MotionDirector: Motion customization of text-to-video diffusion models, European Conference on Computer Vision (ECCV), 2024. https://arxiv.org/abs/2310.08465

[39] Yuwei Guo and others. https://huggingface.co/guoyww/animatediff-motion-adapter-v1-5-2, Accessed on: 20th October, 2026.

[40] Yuwei Guo and others. https://huggingface.co/guoyww/animatediff-motion-adapter-v1-5-3, Accessed on: 26th October, 2026.

[41] emilianJR. https://huggingface.co/emilianJR/epiCRealism, Accessed on: 26th October, 2026.

[42] Hugging Face. https://huggingface.co/docs/diffusers, Accessed on: 20th October, 2026.

[43] DIVM. https://huggingface.co/datasets/divm/hmdb51, Accessed on: 20th October, 2026.